Movie IQ APP

Predictive Analytics on Film Success

A Technical Project Report

Prepared by: Vivek Yadav

Tools: Python · Pandas · scikit-learn · SciPy · Seaborn · Streamlit

PROJECT STREAMLIT APP LINK

CLICK HERE:- APP_LINK

1. Problem Statement

MovieIQ is an interactive analytics and machine-learning project that studies what drives a movie's box-office success and builds a model to predict it ahead of release.

1.1 Definition of Success

A movie is labeled a success when its revenue exceeds its production budget:

success = 1 if revenue > budget, else 0

This is a simplification — it ignores marketing spend, theatrical vs. streaming splits, and inflation-adjusted comparisons across years — but it gives a consistent, reproducible label across the full dataset.

1.2 Why This Matters

- Studios can use success prediction to inform greenlight decisions before committing a full production budget.
- Investors and financiers can use it to assess the risk profile of a slate of films before funding.

1.3 Objective

Build a data-driven system that explains and predicts movie success using budget, popularity, runtime, and audience rating. Concrete steps:

- Clean and prepare the raw movie dataset, engineering a binary success label.
- Explore relationships between budget, revenue, genre, and success through visual analysis.
- Statistically test whether popularity and genre are meaningfully associated with success.
- Train and evaluate a Random Forest classifier to predict success from movie attributes.
- Package the analysis into an interactive Streamlit dashboard for non-technical stakeholders.

1.4 Problem Type

This is a binary classification problem: the target variable is success (1 = successful, 0 = not successful), and the model learns to map input features (budget, popularity, runtime, vote average) to one of these two classes.

2. Data Preparation

The raw dataset (movies.csv) contains 2000 rows across 7 columns: budget, revenue, popularity, runtime, vote_average, title, and genres.

2.1 Cleaning Steps

- Rows with a budget or revenue of 0 were removed a $0 budget or revenue makes the success ratio meaningless (any revenue would trivially "succeed") and would distort both the EDA and the model.
- Rows with missing values in budget, revenue, popularity, runtime, or vote_average were dropped.

After cleaning, 2000 rows remained (no rows were lost in this dataset, as it did not contain zero or missing values in the key columns).

2.2 Target Variable & Class Balance

The success column was computed as revenue > budget. The dataset is imbalanced: 80.7% of movies are labeled successful versus 19.3% unsuccessful. This imbalance was addressed in modeling using class_weight='balanced' in the Random Forest classifier, and by evaluating with precision/recall rather than accuracy alone.

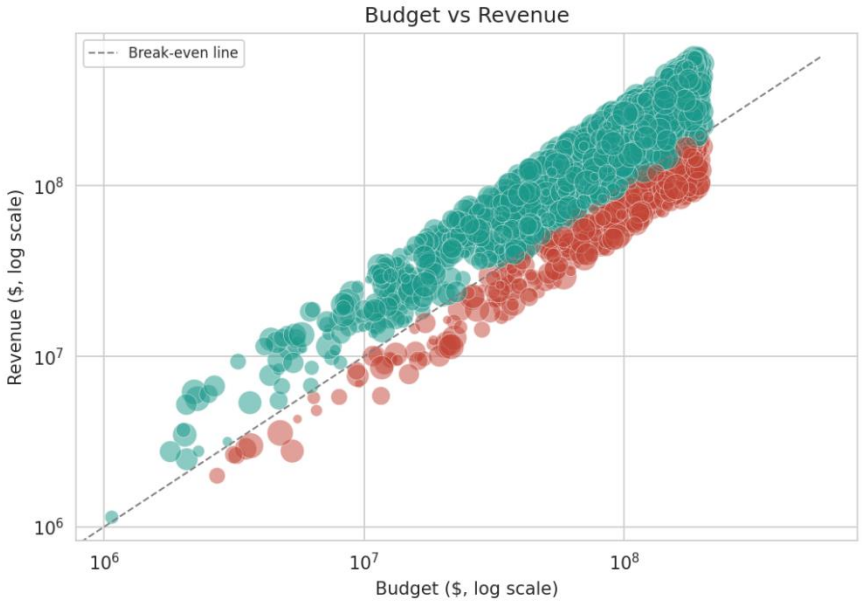
2.3 Genre Processing

The genres column stores a stringified list of dictionaries (e.g. [{'id': 10749, 'name': 'Romance'}]), consistent with TMDB-style datasets. This was parsed with ast.literal_eval() (not json.loads, since the strings use single quotes) and reduced to a plain list of genre names per movie. For genre-level analysis, the dataframe was exploded so that a movie with multiple genres contributes one row per genre.

A derived budget_category field was also engineered Micro/Indie (<$10M), Mid-Budget ($10M–$50M), High-Budget ($50M–$100M), and Blockbuster (>$100M) to support tier-level analysis in the dashboard and this report.

3. Exploratory Data Analysis

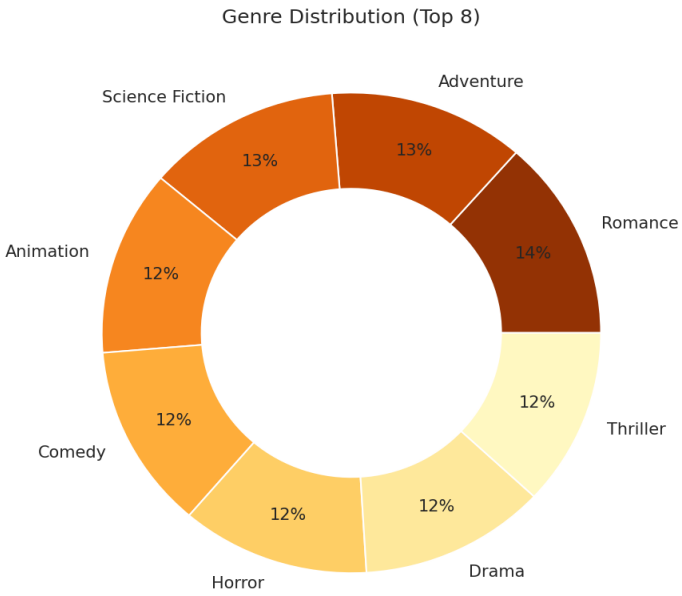
3.1 Budget vs. Revenue



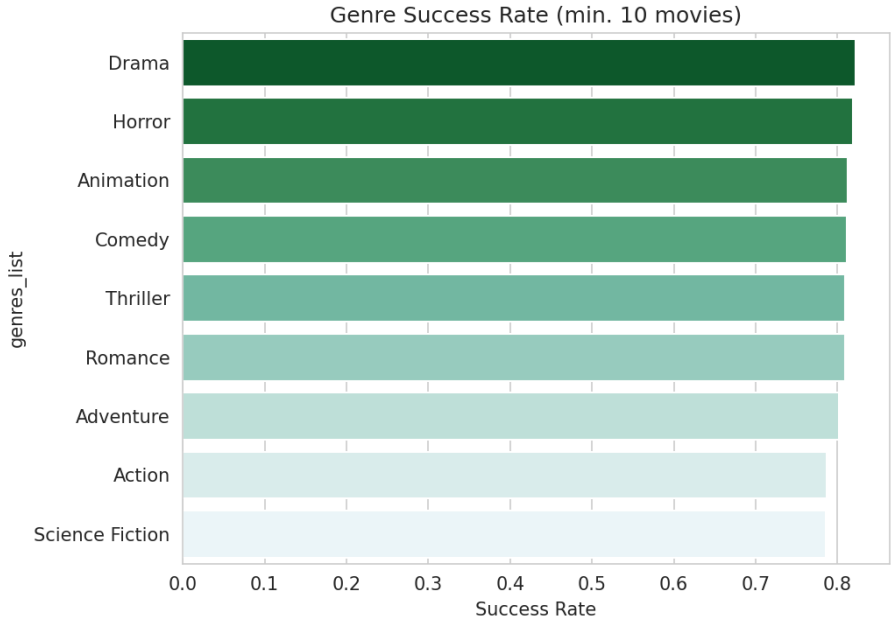
Budget vs. Revenue (log scale). Point size reflects popularity; green = successful, red = not successful. The dashed line marks the break-even point.

Budget and revenue are strongly positively correlated (r = 0.76) higher-budget films tend to earn more in absolute terms. However, the scatter shows many high-budget films sitting close to or below the break-even line, meaning a bigger budget does not reliably translate into profitability once cost is accounted for.

3.2 Genre Trends



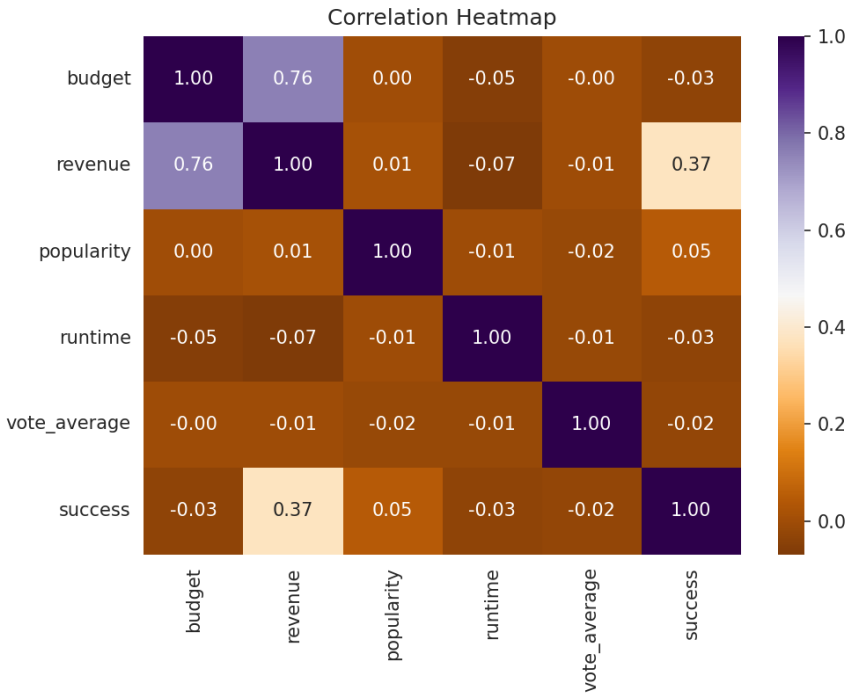
Distribution of the top 8 genres by number of movies.



Success rate by genre (genres with at least 10 movies).

Drama shows the highest success rate among genres with sufficient sample size (82.2%), while Science Fiction shows the lowest (78.5%). The spread between genres is narrow, though — a signal the statistical test in Section 4.2 examines more rigorously.

3.3 Correlation Heatmap



Correlation heatmap of numeric features.

Budget and revenue are the most strongly correlated pair, as expected. Popularity, runtime, and vote_average show weak correlation with success individually — suggesting no single feature dominates, and a multi-feature model is needed to capture the pattern (addressed in Section 5).

4. Statistical Testing

4.1 T-Test — Popularity vs. Success

Null hypothesis: there is no difference in mean popularity between successful and unsuccessful movies.

Mean popularity (successful)	50.73
Mean popularity (unsuccessful)	47.40
T-statistic	2.062
P-value	0.0397

Since p = 0.0397 < 0.05, we reject the null hypothesis: popularity differs significantly between successful and unsuccessful movies, with successful movies showing a modestly higher average popularity score.

4.2 Chi-Square Test — Genre vs. Success

Null hypothesis: genre and movie success are independent (no association).

Chi-square statistic	1.736
Degrees of freedom	8
P-value	0.9881

Here, p = 0.9881 is far above 0.05, so we fail to reject the null hypothesis: in this dataset, genre and success show no statistically significant association among the top 10 genres, despite the visual differences in Figure 3. This is an important caveat — the genre gaps observed in the EDA are not strong enough to be distinguished from random variation at this sample size.

4.3 Interpreting the P-Value

A p-value is the probability of observing a result at least as extreme as the one found, assuming the null hypothesis is true. A common significance threshold is 0.05: below it, a result is considered statistically significant; at or above it, there is insufficient evidence to reject the null hypothesis.

5. Predictive Modeling (Random Forest)

5.1 Features & Target

Features used: budget, popularity, runtime, and vote_average. The title column was excluded as a free-text identifier with no predictive value, and revenue was deliberately excluded because success is directly derived from revenue > budget — including it would leak the target and produce an artificially perfect, non-generalizable model.

5.2 Train/Test Split

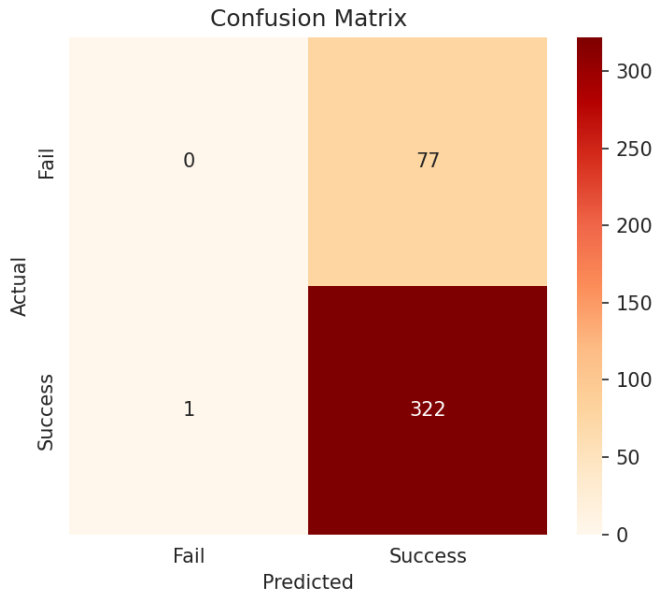
Data was split 80/20 (1600 training rows, 400 test rows) using stratified sampling to preserve the success/failure ratio in both sets. A held-out test set is essential to measure how the model performs on movies it has not seen, rather than how well it memorized the training data.

5.3 How a Random Forest Predicts

A Random Forest builds many individual decision trees, each trained on a random subset of the data and features. Each tree independently predicts success or failure for a given movie, and the forest's final prediction is the majority vote across all trees. This averaging reduces the overfitting risk of any single decision tree and typically improves generalization.

5.4 Evaluation

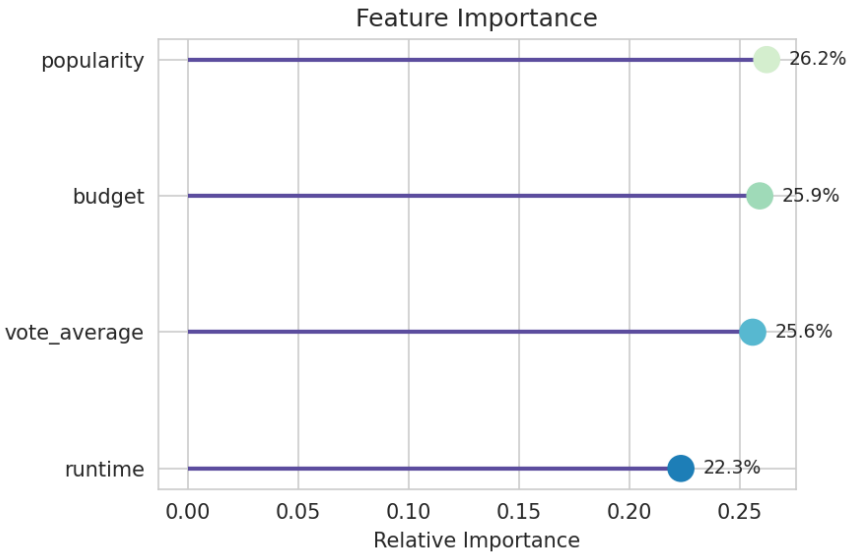
Accuracy	80.5%
Precision	80.7%
Recall	99.7%



Confusion matrix on the held-out test set.

The confusion matrix shows the model correctly identifies almost all successful movies (high recall, 99.7%) but struggles to correctly flag unsuccessful ones — of the 77 actually-unsuccessful movies in the test set, only 0 were correctly identified as such. This is a direct consequence of the class imbalance noted in Section 2.2: even with class_weight='balanced', the model leans toward predicting the majority class (success). In a real greenlight setting, this means MovieIQ in its current form is more useful for flagging likely successes than for reliably catching likely flops.

5.5 Feature Importance



Random Forest feature importances.

popularity emerged as the single strongest predictor (26.2% relative importance), with the remaining three features contributing fairly evenly (all between 22.3% and 26.2%). This roughly agrees with the T-test in Section 4.1, which also found popularity meaningfully associated with success — though the model shows budget and vote_average as nearly equally important, suggesting success is driven by a combination of factors rather than any single dominant one.

6. Interactive Dashboard

The full analysis is wrapped in a Streamlit web application (MovieIQ.py) with sidebar filters for genre, budget range, box-office outcome, runtime, and minimum vote average, plus seven navigable sections:

- Performance Snapshot — genre distribution, budget-vs-revenue, genre success rates, and correlation heatmap.
- Hypothesis Testing — live T-test and Chi-square results that recompute as filters change.
- Prediction Engine — trains and evaluates the Random Forest live, with an adjustable test-set size.
- What-If Predictor — lets a user enter a hypothetical movie's budget, popularity, runtime, and rating to get a live greenlight / hold prediction with confidence score.
- Movie Comparison — side-by-side comparison of any two movies in the filtered dataset.
- Raw Data & Export — a sortable data table with CSV export of the currently filtered movies.
- Insights & Recommendations — auto-generated, filter-aware business insights (see Section 7).

All KPI cards (movie count, average budget, average revenue, success rate, average ROI) update live as filters change, giving stakeholders an at-a-glance view of any slice of the catalogue they choose to explore.

7. Business Insights & Recommendations

7.1 Genre Strategy

Drama shows the strongest success rate among well-represented genres, though Section 4.2 shows this difference is not statistically significant across the top-10 genres as a whole — genre alone is a weak lever for success.

7.2 Budget Strategy

The High-Budget ($50M-$100M) tier shows the strongest success rate (82.9% across 503 movies) in this dataset. Combined with the strong budget-revenue correlation, this suggests a "sweet spot" tier exists where spend is high enough to support strong production and marketing, without the outsized cost base of a blockbuster tentpole.

7.3 Model-Informed Priorities

Because popularity, budget, and vote_average carry comparable predictive weight, greenlight discussions should weigh pre-release buzz (popularity), budget discipline, and anticipated audience reception together, rather than over-indexing on any single metric.

7.4 Summary Recommendation

MovieIQ is best used as a directional decision-support tool, not a final arbiter: it is considerably better at confirming a likely hit than at catching a likely flop, and genre-level effects should be treated as suggestive rather than statistically proven. It is most useful when its outputs are combined with human judgment on script quality, cast, marketing plan, and release-window competition — factors the current dataset does not capture.

8. Reflection

If a studio asked "Will our next film succeed?", MovieIQ's answer should be trusted with moderate confidence for identifying likely successes (99.7% recall), but treated cautiously for ruling out failure — the model rarely predicts "not successful" even for movies that actually failed. A key limitation is that the dataset only includes budget, popularity, runtime, and rating — it has no information on marketing spend, star power, competitive release timing, franchise/sequel status, or critical reviews, all of which materially affect real-world box-office outcomes. With more time and data, the highest-value improvement would be sourcing additional features (marketing budget, franchise flag, release month/season, and star/director track record) and rebalancing the training data — e.g., via oversampling the minority (unsuccessful) class — to improve the model's ability to catch likely flops rather than defaulting toward predicting success.

Appendix A: Full Source Code (MovieIQ.py)

The complete Streamlit application source code is reproduced below for reference and reproducibility.

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import ast
from scipy import stats as sstats
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix

st.set_page_config(page_title="MovieIQ", page_icon="🎬", layout="wide")

# CUSTOM CSS
st.markdown("""
<style>
.hero {
    background: linear-gradient(135deg, #12343b 0%, #1b998b 55%, #2b7a78 100%);
    padding: 28px 32px;
    border-radius: 14px;
    color: white;
    margin-bottom: 20px;
}
.hero h1 { margin: 0; font-size: 26px; }
.hero p { margin: 6px 0 0 0; opacity: 0.9; font-size: 15px; }
.kpi-card {
    padding: 18px 16px;
    border-radius: 12px;
    color: white;
    text-align: left;
}
.kpi-value { font-size: 26px; font-weight: 700; }
.kpi-label { font-size: 13px; opacity: 0.9; }
.filter-header {
    padding: 6px 10px;
    border-radius: 6px;
    color: white;
    font-size: 13px;
    font-weight: 600;
    margin: 14px 0 6px 0;
    display: inline-block;
}
</style>
""", unsafe_allow_html=True)

# LOAD & CLEAN DATA
@st.cache_data
def load_data():
    df = pd.read_csv('movies.csv')
    df = df[(df['budget'] > 0) & (df['revenue'] > 0)].copy()
    df = df.dropna(subset=['budget', 'revenue', 'popularity', 'runtime', 'vote_average'])
    df['success'] = (df['revenue'] > df['budget']).astype(int)
    df['roi'] = (df['revenue'] - df['budget']) / df['budget']

    def extract_genres(g):
        try:
            return [x['name'] for x in ast.literal_eval(g)]
        except (ValueError, SyntaxError):
```

```
            return []
    df['genres_list'] = df['genres'].apply(extract_genres)
    return df

df = load_data()

# HERO BANNER
st.markdown("""
<div class="hero">
    <h1>🎬 MovieIQ — Studio Decision Intelligence</h1>
    <p>Analyze. Predict. Decide with confidence.</p>
</div>
""", unsafe_allow_html=True)

# SIDEBAR — NAVIGATION (replaces tabs)
st.sidebar.markdown('<div class="filter-header" style="background:#12343b;">🧭 NAVIGATE</div>',
unsafe_allow_html=True)
page = st.sidebar.radio(
    "Navigate",
    ["📊 Performance Snapshot", "🔬 Hypothesis Testing", "🔮 Prediction Engine",
     "🎯 What-If Predictor", "⚖️ Movie Comparison", "📋 Raw Data & Export",
     "💡 Insights & Recommendations"],
    label_visibility="collapsed"
)

st.sidebar.divider()
st.sidebar.markdown('<div class="filter-header" style="background:#1b998b;">🎭 GENRES</div>',
unsafe_allow_html=True)
all_genres = sorted(set(g for genres in df['genres_list'] for g in genres))
selected_genres = st.sidebar.multiselect("Select Genres", all_genres, default=[],
label_visibility="collapsed")

st.sidebar.markdown('<div class="filter-header" style="background:#5c4d9e;">💰 BUDGET RANGE</div>',
unsafe_allow_html=True)
min_b, max_b = int(df['budget'].min()), int(df['budget'].max())
budget_range = st.sidebar.slider(
    "Budget Range", min_b, max_b, (min_b, max_b),
    step=1_000_000, format="$%d", label_visibility="collapsed"
)

st.sidebar.markdown('<div class="filter-header" style="background:#c44536;">🎟️ BOX OFFICE OUTCOME</div>',
unsafe_allow_html=True)
outcome = st.sidebar.radio("Box Office Outcome", ["All Movies", "Successful Only", "Unsuccessful Only"],
label_visibility="collapsed")

st.sidebar.markdown('<div class="filter-header" style="background:#e0a458;">⏱️ RUNTIME RANGE</div>',
unsafe_allow_html=True)
min_runtime, max_runtime = int(df['runtime'].min()), int(df['runtime'].max())
runtime_range = st.sidebar.slider("Runtime Range (minutes)", min_runtime, max_runtime, (min_runtime,
max_runtime), label_visibility="collapsed")

st.sidebar.markdown('<div class="filter-header" style="background:#2b7a78;">⭐ MINIMUM VOTE AVERAGE</div>',
unsafe_allow_html=True)
min_vote = st.sidebar.slider("Minimum vote average", 0.0, 10.0, 0.0, 0.1, label_visibility="collapsed")

# APPLY FILTERS
filtered = df[
    (df['vote_average'] >= min_vote) &
    (df['runtime'] >= runtime_range[0]) &
    (df['runtime'] <= runtime_range[1]) &
    (df['budget'] >= budget_range[0]) &
    (df['budget'] <= budget_range[1])
].copy()
```

```
if selected_genres:
    filtered = filtered[filtered['genres_list'].apply(lambda x: any(g in x for g in selected_genres))]

if outcome == "Successful Only":
    filtered = filtered[filtered['success'] == 1]
elif outcome == "Unsuccessful Only":
    filtered = filtered[filtered['success'] == 0]

if len(filtered) == 0:
    st.warning("No movies match the current filters. Try widening your selection.")
    st.stop()

# KPI CARDS (unchanged)
avg_budget = filtered['budget'].mean()
avg_revenue = filtered['revenue'].mean()
success_rate = filtered['success'].mean()
avg_roi = filtered['roi'].mean()

k1, k2, k3, k4, k5 = st.columns(5)
kpi_data = [
    (k1, "#1b998b", f"{len(filtered)}", "Movies in View"),
    (k2, "#5c4d9e", f"${avg_budget/1e6:.1f}M", "Avg Budget"),
    (k3, "#c44536", f"${avg_revenue/1e6:.1f}M", "Avg Revenue"),
    (k4, "#e0a458", f"{success_rate:.1%}", "Success Rate"),
    (k5, "#2b7a78", f"{avg_roi:.1%}", "Avg ROI"),
]
for col, color, value, label in kpi_data:
    col.markdown(f"""
    <div class="kpi-card" style="background:{color};">
        <div class="kpi-value">{value}</div>
        <div class="kpi-label">{label}</div>
    </div>
    """, unsafe_allow_html=True)

st.write("")

# PAGE: PERFORMANCE SNAPSHOT
if page == "🎞 Performance Snapshot":
    c1, c2 = st.columns(2)

    with c1:
        st.subheader("Top Genres (Share of Movies)")
        exploded = filtered.explode('genres_list')
        top_genres = exploded['genres_list'].value_counts().head(8)
        fig, ax = plt.subplots(figsize=(6, 6))
        colors = sns.color_palette('YlOrBr_r', len(top_genres))
        wedges, texts, autotexts = ax.pie(
            top_genres.values, labels=top_genres.index, autopct='%1.0f%%',
            colors=colors, pctdistance=0.8,
            wedgeprops=dict(width=0.4, edgecolor='white')
        )
        ax.set_title("Genre Distribution (Donut)")
        st.pyplot(fig)

    with c2:
        st.subheader("Budget vs Revenue")
        fig, ax = plt.subplots(figsize=(6, 6))
        sizes = (filtered['popularity'] / filtered['popularity'].max()) * 200 + 20
        colors = filtered['success'].map({0: '#c44536', 1: '#1b998b'})
        ax.scatter(filtered['budget'], filtered['revenue'], s=sizes, c=colors, alpha=0.55,
edgecolors='white', linewidth=0.5)
```

```
        # break-even reference line (revenue = budget)
        line_max = max(filtered['budget'].max(), filtered['revenue'].max())
        ax.plot([0, line_max], [0, line_max], color='gray', linestyle='--', linewidth=1, label='Break-even
line')

        ax.set_xscale('log')
        ax.set_yscale('log')
        ax.set_xlabel("Budget ($, log scale)")
        ax.set_ylabel("Revenue ($, log scale)")
        ax.legend(loc='upper left', fontsize=8)
        st.caption("Point size = popularity · Green = success, Red = not successful")
        st.pyplot(fig)

    st.divider()

    c3, c4 = st.columns(2)
    with c3:
        st.subheader("Genre Success Rate")
        genre_success =
exploded.groupby('genres_list')['success'].mean().sort_values(ascending=False).head(10)
        fig, ax = plt.subplots()
        sns.barplot(x=genre_success.values, y=genre_success.index, hue=genre_success.index,
                    palette='BuGn_r', legend=False, ax=ax)
        st.pyplot(fig)

    with c4:
        st.subheader("Top 10 Revenue Movies")
        top10 = filtered.nlargest(10, 'revenue')[['title', 'revenue']].sort_values('revenue')
        fig, ax = plt.subplots()
        sns.barplot(x=top10['revenue'], y=top10['title'], hue=top10['title'],
                    palette='flare', legend=False, ax=ax)
        ax.set_xlabel("Revenue ($)")
        ax.set_ylabel("")
        st.pyplot(fig)

    st.subheader("Correlation Heatmap")
    numeric_cols = ['budget', 'revenue', 'popularity', 'runtime', 'vote_average', 'success']
    fig, ax = plt.subplots(figsize=(8, 5))
    sns.heatmap(filtered[numeric_cols].corr(), annot=True, cmap='PuOr', fmt='.2f', ax=ax)
    st.pyplot(fig)

# PAGE: HYPOTHESIS TESTING
elif page == "🧪 Hypothesis Testing":
    st.subheader("T-Test: Popularity vs Success")
    success_pop = filtered[filtered['success'] == 1]['popularity']
    fail_pop = filtered[filtered['success'] == 0]['popularity']

    if len(success_pop) > 1 and len(fail_pop) > 1:
        t_stat, p_val = sstats.ttest_ind(success_pop, fail_pop, equal_var=False)
        c1, c2 = st.columns(2)
        c1.metric("T-statistic", f"{t_stat:.3f}")
        c2.metric("P-value", f"{p_val:.4f}")
        st.write("**Null hypothesis:** No difference in mean popularity between successful and
unsuccessful movies.")
        if p_val < 0.05:
            st.success(f"p-value ({p_val:.4f}) < 0.05 → Reject null hypothesis. Popularity differs
significantly.")
        else:
            st.warning(f"p-value ({p_val:.4f}) ≥ 0.05 → Fail to reject null hypothesis.")
    else:
        st.info("Not enough data in both groups under current filters to run a T-test.")

    st.divider()
```

```
    st.subheader("Chi-Square Test: Genre vs Success")
    exploded_chi = filtered.explode('genres_list')
    top10_genres = exploded_chi['genres_list'].value_counts().head(10).index
    chi_data = exploded_chi[exploded_chi['genres_list'].isin(top10_genres)]
    contingency = pd.crosstab(chi_data['genres_list'], chi_data['success'])

    if contingency.shape[0] > 1 and contingency.shape[1] > 1:
        chi2, p_val_chi, dof, expected = sstats.chi2_contingency(contingency)
        c1, c2, c3 = st.columns(3)
        c1.metric("Chi-square", f"{chi2:.3f}")
        c2.metric("P-value", f"{p_val_chi:.4f}")
        c3.metric("Degrees of freedom", dof)
        st.write("**Null hypothesis:** Genre and success are independent.")
        if p_val_chi < 0.05:
            st.success(f"p-value ({p_val_chi:.4f}) < 0.05 → Reject null hypothesis. Genre and success are
associated.")
        else:
            st.warning(f"p-value ({p_val_chi:.4f}) ≥ 0.05 → Fail to reject null hypothesis.")
        with st.expander("View contingency table"):
            st.dataframe(contingency)
    else:
        st.info("Not enough variation under current filters to run a Chi-square test.")

    st.info("**P-value:** probability of this result (or more extreme) if the null hypothesis were true.
Threshold used: 0.05.")

# PAGE: PREDICTION ENGINE
elif page == "🤖 Prediction Engine":
    st.subheader("Random Forest Classifier")
    feature_cols = ['budget', 'popularity', 'runtime', 'vote_average']
    X, y = df[feature_cols], df['success']

    test_size = st.slider("Test set size", 0.1, 0.4, 0.2, 0.05)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=42,
stratify=y)

    @st.cache_resource
    def train_model(X_train, y_train, _test_size):
        model = RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42)
        model.fit(X_train, y_train)
        return model

    model = train_model(X_train, y_train, test_size)
    y_pred = model.predict(X_test)

    c1, c2, c3 = st.columns(3)
    c1.metric("Accuracy", f"{accuracy_score(y_test, y_pred):.2%}")
    c2.metric("Precision", f"{precision_score(y_test, y_pred):.2%}")
    c3.metric("Recall", f"{recall_score(y_test, y_pred):.2%}")
    st.caption("Excluded `title` (not predictive) and `revenue` (would leak the target).")

    c4, c5 = st.columns(2)
    with c4:
        st.subheader("Confusion Matrix")
        cm = confusion_matrix(y_test, y_pred)
        fig, ax = plt.subplots(figsize=(5, 4))
        sns.heatmap(cm, annot=True, fmt='d', cmap='OrRd',
                    xticklabels=['Fail', 'Success'], yticklabels=['Fail', 'Success'], ax=ax)
        st.pyplot(fig)

    with c5:
        st.subheader("Feature Importance (Lollipop)")
```

```
        importances = pd.Series(model.feature_importances_,
index=feature_cols).sort_values(ascending=True)
        fig, ax = plt.subplots(figsize=(5, 4))
        colors = sns.color_palette('GnBu_r', len(importances))
        ax.hlines(y=importances.index, xmin=0, xmax=importances.values, color='#5c4d9e', linewidth=2)
        ax.scatter(importances.values, importances.index, color=colors, s=150, zorder=3)
        for i, v in enumerate(importances.values):
            ax.text(v + 0.01, i, f"{v:.1%}", va='center', fontsize=9)
        ax.set_xlabel("Relative Importance")
        st.pyplot(fig)
        importances = importances.sort_values(ascending=False)  # keep descending order for downstream
tabs

# PAGE: WHAT-IF PREDICTOR
elif page == "🔮 What-If Predictor":
    st.subheader("🔮 What-If Predictor")
    st.write("Enter a hypothetical movie's details to simulate a success prediction.")

    feature_cols = ['budget', 'popularity', 'runtime', 'vote_average']
    X, y = df[feature_cols], df['success']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

    @st.cache_resource
    def train_default_model(X_train, y_train):
        model = RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42)
        model.fit(X_train, y_train)
        return model

    sim_model = train_default_model(X_train, y_train)

    c1, c2 = st.columns(2)
    with c1:
        input_budget = st.number_input("Budget ($)", min_value=1000.0, value=50_000_000.0,
step=1_000_000.0)
        input_popularity = st.number_input("Popularity score", min_value=0.0, value=50.0, step=1.0)
    with c2:
        input_runtime = st.number_input("Runtime (minutes)", min_value=1.0, value=120.0, step=1.0)
        input_vote = st.number_input("Vote average (0-10)", min_value=0.0, max_value=10.0, value=6.5,
step=0.1)

    if st.button("Simulate Greenlight Decision", type="primary"):
        input_df = pd.DataFrame([{
            'budget': input_budget, 'popularity': input_popularity,
            'runtime': input_runtime, 'vote_average': input_vote
        }])
        pred = sim_model.predict(input_df)[0]
        proba = sim_model.predict_proba(input_df)[0]

        if pred == 1:
            st.success(f"✅ GREENLIGHT — Predicted Success (confidence: {proba[1]:.1%})")
        else:
            st.error(f"❌ HOLD — Predicted Not Successful (confidence: {proba[0]:.1%})")

# PAGE: MOVIE COMPARISON
elif page == "🎬 Movie Comparison":
    st.subheader("🎬 Movie Comparison")
    st.write("Pick two movies from the current filter selection to compare side by side.")

    titles = sorted(filtered['title'].unique())
    if len(titles) < 2:
        st.info("Need at least 2 movies in the current filter selection to compare.")
    else:
        c1, c2 = st.columns(2)
```

```
        with c1:
            movie_a = st.selectbox("Movie A", titles, index=0)
        with c2:
            movie_b = st.selectbox("Movie B", titles, index=1)

        row_a = filtered[filtered['title'] == movie_a].iloc[0]
        row_b = filtered[filtered['title'] == movie_b].iloc[0]

        col_a, col_b = st.columns(2)

        with col_a:
            st.markdown(f"#### 🎬 {movie_a}")
            st.metric("Budget", f"${row_a['budget']/1e6:.1f}M")
            st.metric("Revenue", f"${row_a['revenue']/1e6:.1f}M")
            st.metric("Popularity", f"{row_a['popularity']:.1f}")
            st.metric("Runtime", f"{row_a['runtime']:.0f} min")
            st.metric("Vote Average", f"{row_a['vote_average']:.1f}")
            st.metric("ROI", f"{row_a['roi']:.1%}")
            status_a = "✅Successful" if row_a['success'] == 1 else "❌Not Successful"
            st.markdown(f"**Outcome:** {status_a}")

        with col_b:
            st.markdown(f"#### 🎬 {movie_b}")
            st.metric("Budget", f"${row_b['budget']/1e6:.1f}M")
            st.metric("Revenue", f"${row_b['revenue']/1e6:.1f}M")
            st.metric("Popularity", f"{row_b['popularity']:.1f}")
            st.metric("Runtime", f"{row_b['runtime']:.0f} min")
            st.metric("Vote Average", f"{row_b['vote_average']:.1f}")
            st.metric("ROI", f"{row_b['roi']:.1%}")
            status_b = "✅Successful" if row_b['success'] == 1 else "❌Not Successful"
            st.markdown(f"**Outcome:** {status_b}")

        st.divider()
        st.subheader("Budget vs Revenue Comparison")
        fig, ax = plt.subplots(figsize=(7, 4))
        x = np.arange(2)
        width = 0.35
        ax.bar(x - width/2, [row_a['budget'], row_b['budget']], width, label='Budget', color='#5c4d9e')
        ax.bar(x + width/2, [row_a['revenue'], row_b['revenue']], width, label='Revenue', color='#1b998b')
        ax.set_xticks(x)
        ax.set_xticklabels([movie_a, movie_b])
        ax.legend()
        st.pyplot(fig)

# PAGE: RAW DATA & EXPORT
elif page == "📋 Raw Data & Export":
    st.subheader("📋 Raw Data & Export")
    st.write(f"Showing {len(filtered)} movies matching current filters.")
    display_df = filtered[['title', 'budget', 'revenue', 'popularity', 'runtime', 'vote_average',
'success']].copy()
    display_df['_sort_key'] = display_df['title'].str.extract(r'(\d+)').astype(float)
    display_df = display_df.sort_values('_sort_key').drop(columns='_sort_key')
    st.dataframe(display_df, use_container_width=True)
    csv = filtered.to_csv(index=False).encode('utf-8')
    st.download_button("Download filtered data as CSV", csv, "movieiq_filtered.csv", "text/csv")

# PAGE: BUSINESS INSIGHTS & RECOMMENDATIONS
elif page == "💡 Insights & Recommendations":
    st.subheader("💡 Business Insights & Recommendations")
    st.caption("Auto-generated from the currently filtered dataset — adjust filters in the sidebar to
update.")

    exploded_ins = filtered.explode('genres_list')
```

```
    genre_counts = exploded_ins['genres_list'].value_counts()
    eligible_genres = genre_counts[genre_counts >= 10].index
    genre_perf = exploded_ins[exploded_ins['genres_list'].isin(eligible_genres)] \
        .groupby('genres_list')['success'].mean().sort_values(ascending=False)

    filtered['runtime_bucket'] = pd.cut(filtered['runtime'], bins=[0, 90, 120, 150, 300],
                                        labels=['<90 min', '90-120 min', '120-150 min', '150+ min'])
    runtime_perf = filtered.groupby('runtime_bucket', observed=True)['success'].agg(['mean',
'count']).sort_values('mean', ascending=False)

    br_corr = filtered['budget'].corr(filtered['revenue'])

    st.markdown("### 🎭 Genre Strategy")
    if len(genre_perf) > 0:
        best_genre = genre_perf.index[0]
        worst_genre = genre_perf.index[-1]
        st.success(f"**{best_genre}** has the highest success rate ({genre_perf.iloc[0]:.1%}) among genres
with sufficient sample size — prioritize slates leaning into this genre.")
        st.warning(f"**{worst_genre}** has the lowest success rate ({genre_perf.iloc[-1]:.1%}) in the
current selection — treat projects in this genre with extra scrutiny.")
    else:
        st.info("Not enough genre data in the current filter to draw a genre recommendation.")

    st.markdown("### 💰 Budget Strategy")
    st.write(f"Budget and revenue correlation in this selection: **r = {br_corr:.2f}**.")
    if br_corr > 0.6:
        st.write("Bigger budgets tend to track with bigger revenue, but that doesn't guarantee
**profitability** (success = revenue > budget).")
    else:
        st.write("Spending more doesn't reliably translate to proportionally higher revenue here.")

    st.markdown("### ⏱ Runtime Strategy")
    if len(runtime_perf) > 0:
        best_runtime = runtime_perf['mean'].idxmax()
        st.success(f"Movies in the **{best_runtime}** range have the highest success rate
({runtime_perf.loc[best_runtime, 'mean']:.1%}) in this selection.")
    else:
        st.info("Not enough data to compare runtime buckets.")

    st.divider()
    st.markdown("### 🧭 Summary Recommendation")
    st.write(
        "Combine genre, budget, and runtime signals rather than any single factor in isolation — "
        "the statistical tests (see **Hypothesis Testing**) confirm some of these relationships are "
        "significant, but correlation here reflects historical patterns in this dataset, not guaranteed "
        "future outcomes. Use the **What-If Predictor** to sanity-check a specific project against the
model."
    )
```